

Structures Conditionnelles

Essentielles dans tout langage de programmation, les structures conditionnelles permettent d'exécuter différentes parties de code en fonction de conditions spécifiques. En JavaScript, les principales structures conditionnelles sont `if`, `else if`, `else` et `switch`. Cela permet de contrôler le fil d'exécution du programme en fonction des valeurs des variables ou des résultats d'expressions (conditions).

Si une condition est vraie, un bloc de code spécifique est exécuté. Si elle est fausse, un autre bloc de code peut être exécuté (via `else` ou `else if`). Le `switch` est utilisé pour comparer une variable à plusieurs valeurs possibles et exécuter le bloc de code correspondant.

Structure `if`, `else if`, `else`

La structure `if` permet d'exécuter un bloc de code si une condition est vraie. Vous pouvez également utiliser `else if` pour vérifier d'autres conditions si la première est fausse, et `else` pour exécuter un bloc de code si toutes les conditions précédentes sont fausses.

Vous devez obligatoirement commencer par un `if`. Vous pouvez ajouter autant de `else if` que nécessaire mais c'est optionnel. Le `else` est optionnel également et ne peut apparaître qu'une seule fois à la fin.

Le premier bloc dont la condition est vraie sera exécuté, les autres seront ignorés.

En pseudo-code, on écrira:

```
SI condition1
    // actions à faire
SINON SI condition2
    // actions à faire
SINON SI condition3
    // actions à faire
SINON
    // actions à faire
FIN SI
```

```
let age = 20;
let estMajeur = age >= 18; // true si l'âge est 18 ou plus, sinon false

//Vous pouvez utiliser un if seul
if (estMajeur) {
    console.log("Vous êtes majeur.");
}

//Si vous avez besoin de vérifier plusieurs conditions, vous pouvez utiliser if,
else if et else
//Le premier bloc dont la condition est vraie sera exécuté, les autres seront
ignorés
if (age < 13) {
    //Si l'âge est inférieur à 13
```

```

    console.log("Vous êtes un enfant.");
} else if (age >= 13 && age < 20) {
    //Si l'âge est entre 13 et 19 inclusivement
    console.log("Vous êtes un adolescent.");
} else {
    //Si aucune des conditions précédentes n'est vraie, ce bloc sera
    automatiquement exécuté
    console.log("Vous êtes un adulte.");
}

```

Imbrication de structures conditionnelles

Il est possible d'imbriquer des structures conditionnelles à l'intérieur d'autres structures conditionnelles mais cela peut rendre le code difficile à lire et à maintenir. À éviter si possible.

Il est souvent préférable d'utiliser des opérateurs logiques pour combiner plusieurs conditions dans une seule structure conditionnelle. (Voir section plus bas)

```

let nAge = 20;
let sNom = "Alice";
let bEstMajeur = age >= 18; // true si l'âge est 18
let bNomEstAlice = sNom === "Alice"; // true si le nom est "Alice"

//Exemple d'imbrication de structures conditionnelles, plus que 2 niveaux
d'imbrication est à éviter
//Si la personne est majeure
if (bEstMajeur) {
    //Si le nom est Alice
    if (bNomEstAlice) {
        console.log("Bienvenue, Alice, vous êtes majeur.");
    } else {
        console.log("Bienvenue, vous êtes majeur mais vous n'êtes pas Alice.");
    }
} else {
    console.log("Vous n'êtes pas majeur.");
}

//Exemple utilisant des opérateurs logiques pour combiner les conditions lorsque
possible
if (bEstMajeur && bNomEstAlice) {
    //Si la personne est majeure ET que le nom est Alice
    console.log("Bienvenue, Alice, vous êtes majeur.");
} else if (bEstMajeur && !bNomEstAlice) {
    //Si la personne est majeure ET que le nom n'est PAS Alice
    console.log("Bienvenue, vous êtes majeur. Qui êtes-vous ?");
} else {
    console.log("Vous n'êtes pas majeur.");
}

```

Opérateurs logiques

Les opérateurs logiques permettent de combiner plusieurs conditions dans une seule expression conditionnelle. Les principaux opérateurs logiques en JavaScript sont :

- **&&** (ET logique) : Retourne vrai seulement si les deux conditions sont vraies.
- **||** (OU logique) : Retourne vrai si au moins une des conditions est vraie.
- **!** (NON logique) : Inverse la valeur de vérité d'une condition (vrai devient faux et vice versa).

```
let nTemps = 100;
let nPointsVie = 10;
let bJeuTermine = false;
let bAfficherMessageFin = false;

//Vérifie si le joueur est en vie ET que le jeu n'est pas terminé
if (nTemps > 0 && nPointsVie > 0) {
    bJeuTermine = false;
    console.log("Le joueur est en vie et le jeu continue.");
} else if (nTemps <= 0 || nPointsVie <= 0) {
    //Vérifie si le temps est écoulé OU que les points de vie sont à 0 ou moins
    bJeuTermine = true;
    console.log("Le joueur est mort ou le temps est écoulé. Le jeu est terminé.");
}

// C'est la même chose que vérifier si bJeuTermine == false
if (!bJeuTermine) {
    //Vérifie si le jeu n'est PAS terminé
    console.log("Le jeu est toujours en cours.");
}

// Même chose que vérifier si bJeuTermine == true
if (bJeuTermine) {
    //Vérifie si le jeu est terminé
    bAfficherMessageFin = !bAfficherMessageFin; // Inverse la valeur de
    bAfficherMessageFin et la réenregistre dans la variable
}
```

Table de vérité des opérateurs logiques

La table de vérité des opérateurs logiques est un outil utile pour comprendre comment les opérateurs **&&**, **||** et **!** fonctionnent.

Si l'on compare deux valeurs booléennes A et B, la table de vérité est la suivante : Ex: si A est vrai et B est vrai, alors:

- **A && B** est vrai (les deux conditions sont vraies)
- **A || B** est vrai (au moins une condition est vraie)
- **!A** est faux (A est vrai, donc son inverse est faux)
- **!B** est faux (B est vrai, donc son inverse est faux)

A	B	A && B	A B	!A	!B
-----	-----	-----	-----	-----	-----

true	true	true	true	false	false	
true	false	false	true	false	true	
false	true	false	true	true	false	
false	false	false	false	true	true	

Structure switch

La structure `switch` est utilisée pour comparer une variable à plusieurs valeurs possibles. Elle est souvent utilisée lorsque vous avez plusieurs conditions à vérifier pour une même variable, ce qui peut rendre le code plus lisible que d'utiliser plusieurs `if` et `else if`.

Elle s'écrit en commençant par le mot-clé `switch`, suivi de la variable à comparer entre parenthèses.

Ensuite, vous utilisez le mot-clé `case` pour chaque valeur possible de la variable. Si la variable correspond à une des valeurs, le bloc de code associé à ce `case` est exécuté. Vous pouvez utiliser `break` pour sortir du `switch` après l'exécution du bloc de code correspondant, c'est-à-dire que le code ne continuera pas à exécuter les `case` suivants.

Si aucune des valeurs ne correspond, vous pouvez utiliser le mot-clé `default` pour exécuter un bloc de code par défaut. Le bloc `default` est optionnel et sera exécuté si aucune des conditions précédentes n'est vraie.

En pseudo-code, cela se présente ainsi:

```
SWITCH (variable) {
  CAS valeur1:
    // tâche à effectuer
    break;
  CAS valeur2:
    // tâche à effectuer
    break;
  default:
    // bloc de code si aucune des valeurs ne correspond
}
```

En JavaScript, cela reste similaire,

```
let nJour = 3; // Exemple de jour de la semaine (1 = Lundi, 2 = Mardi, etc.)
let sJourSemaine;

switch (nJour) {
  case 1:
    sJourSemaine = "Lundi";
    break;
  case 2:
    sJourSemaine = "Mardi";
    break;
  case 3:
    sJourSemaine = "Mercredi";
    break;
  case 4:
```

```

        sJourSemaine = "Jeudi";
        break;
    case 5:
        sJourSemaine = "Vendredi";
        break;
    case 6:
        sJourSemaine = "Samedi";
        break;
    case 7:
        sJourSemaine = "Dimanche";
        break;
    default:
        sJourSemaine = "Inconnu"; // Si nJour n'est pas entre 1 et 7
}

// Exemple d'utilisation du switch pour gérer les touches du clavier
let sToucheClavier = "A"; // Exemple de touche du clavier
let sDirectionPersonnage;

switch (sToucheClavier) {
    case "W":
        sDirectionPersonnage = "Haut";
        break;
    case "A":
        sDirectionPersonnage = "Gauche";
        break;
    case "S":
        sDirectionPersonnage = "Bas";
        break;
    case "D":
        sDirectionPersonnage = "Droite";
        break;
    default:
        sDirectionPersonnage = "Inconnu"; // Si la touche n'est pas W, A, S ou D
}

```